Gnosis Inventory System – Requirements (v1.6)

First use case: NuePrint print shop operations | Last updated: 2026-06-21

1. Purpose

Build a web-based inventory system for a full-service print shop to track materials, consumables, and (optionally) finished goods; support job-based allocation and picking; keep counts accurate through receiving, consumption, and cycle counts; integrate with Jira; and produce actionable reporting for operations and purchasing.

2. Goals

- Know what is on-hand, where it is, and what is committed (allocated) to jobs/clients.
- Speed up receiving, picking, and counting using barcode/QR scanning and mobile-friendly screens.
- Reduce production delays with pick lists per Jira job and early shortage detection.
- Support purchasing decisions with reorder points, vendor pricing, and low-stock alerts.
- Maintain accountability with an append-only transaction ledger and complete audit history.

3. Confirmed Decisions (Locked)

- Allocations are required (inventory can be committed to a Jira job/client and removed from available stock).
- Roll tracking uses Remaining Length as the stored quantity; remaining length can be updated via measurement methods.
- Paper is tracked as Sleeves and Sheets (sleeves as pack unit; sheets as base unit).
- Multi-location and bin support is included (even if only one location is used initially).
- Support BOM/Templates to standardize material requirements for repeat job types.
- Support item pricing by vendor, including multiple vendors per same item at different costs.
- Support manufacturer/vendor barcodes where reliable, and internal SKU/unit labels where required.
- Role-based access control (RBAC) will be implemented as a separate Auth/RBAC module or service and integrated with the inventory API (see Section 4).
- Items must be organized into categories (hierarchical), with category-based filtering and reporting (see Section 6).

4. Users, Roles, and Access Control

4.1 Roles (Logical)

These are the logical roles the system must support. The inventory application/API should not hard-code roles; instead it should validate capabilities/permissions from the separate Auth/RBAC module.

- Admin: system settings, integrations, master data (items, vendors, locations, categories).
- Inventory Manager: receiving, adjustments, vendor pricing, reorder settings, reporting.
- Production Lead: job material requirements, pick lists, shortage resolution, staging.
- Production Staff: pick/scan/consume, cycle counts; limited adjustments.
- Purchasing (optional): low-stock review, vendor management, PO creation in Phase 2.

4.2 Access Control Architecture (Separate Auth/RBAC Module + Integration)

- A separate Auth/RBAC module or service will provide roles, permissions/capabilities, and user assignment.
- The inventory API must define a set of inventory capabilities (e.g., gnosis_inventory_receive, gnosis_inventory_adjust, gnosis_inventory_count, gnosis_inventory_report, gnosis_inventory_admin).
- The separate Auth/RBAC module or service assigns those capabilities to roles and users; the inventory API only validates capabilities and enforces access decisions server-side.
- If the Auth/RBAC service is unavailable, the inventory system must fail closed: only platform administrators or an emergency super-admin role can access inventory functions.

4.3 Audit and Approvals (Minimum)

- All inventory-changing actions must record: user, timestamp, action type, before/after quantities, and reason code (where applicable).
- Adjustments must require a reason code; optional approval workflow for high-value adjustments should be configurable (Phase 2 optional).

5. Inventory Data Model (Core)

5.1 Item Types

- Raw Materials: roll media, paper (sleeves/sheets), rigid sheets (4×8, etc.).
- Consumables: inks, cleaning solutions, blades, gloves, tape, maintenance kits.
- Finished Goods (optional Phase 1): stocked sellable items.
- Assets / Machines (optional Phase 2): non-consumable equipment tracking (serial, location, maintenance), if desired.

5.2 Tracking Modes

- Unit-tracked: specific physical units must be tracked (required for rolls; recommended for lot/expiry items).

- Bulk-tracked: tracked by quantity in a bin (paper sleeves/sheets, many consumables).

5.3 Units of Measure (UOM) and Conversions

The system must support UOMs and conversion rules appropriate for print production:

- Each (EA), Box/Case, Carton, Sleeve, Sheet, Roll, Linear Feet (LF) (or Yards), Square Feet (SF).
- Paper conversions: sheets-per-sleeve (required); optional sleeves-per-carton.
- Roll conversions: store remaining length; optionally compute remaining square feet using roll width.

5.4 Locations and Bins

- Multiple locations (Warehouse, Print Room, Staging, Shipping, Offsite).
- Bins within locations (aisle/rack/shelf/bin optional).
- Transfers between locations/bins must be recorded as ledger transactions.

6. Categories and Classification (Required)

Items must be organized into categories to support usability, reporting, reorder planning, and future permission scoping.

6.1 Category Model

- Hierarchical categories (parent/child). Example top-level: Paper, Roll Media, Rigid Sheets, Inks, Maintenance, Consumables, Packaging, Tools, Machines/Assets.
- Each item must have exactly one primary category; optional secondary tags may be added later.
- Categories must be manageable via an admin screen (add/edit/merge/disable).

6.2 Category-Driven UX and Reporting

- Inventory search and lists must filter by category and location.
- Reports must support category filtering (low stock by category, usage by category, waste by category).
- Optional future: category-based reorder policies (different reorder logic by category).
- Optional future: category-based access control (e.g., only certain roles can adjust inks).

7. Vendors and Pricing (Required)

7.1 Vendor Master

- Vendor name, contact info (optional), typical lead time (optional).

7.2 Item-Vendor Pricing (Many-to-Many)

- Support multiple vendors per internal item/SKU with different costs and vendor part numbers.

- Store vendor purchasing unit (case/carton/roll) and conversion to internal usage units.
- Allow preferred vendor flag and effective dates (optional).
- On receiving, selecting vendor should auto-fill last known cost (editable with audit trail).

7.3 Costing (Phase 1 Recommendation)

- Store unit cost on receipts to enable cost consumed per job/client and basic valuation reporting.
- Use Average Cost or Last Cost in Phase 1 (configurable); FIFO can be added later if needed.

8. Barcode and Labeling Strategy (Required)

8.1 Supported Barcode Types

- Manufacturer/Vendor barcode (UPC/EAN/GTIN): scan to identify item when reliable.
- Internal item barcode (SKU label): scan to identify item type (often placed on bin/shelf).
- Internal unit barcode (unique unit ID): scan to identify a specific unit (required for rolls).

8.2 Scan Resolution Priority

- If scan matches a Unit ID: open that specific unit record (roll/lot unit).
- Else if scan matches an Item barcode (internal or mapped vendor barcode): open the item record for actions (receive/pick/consume).
- Else: unknown barcode; authorized roles can map barcode to an item.

8.3 Labels and Printing

- On receiving, the system must be able to generate internal labels (QR/barcode) and print them in batch.
- Label content (minimum): unit ID (or SKU), item name, width/thickness (if roll), vendor, received date, location/bin.
- If a unit is allocated, label must optionally show: Allocated Client, Jira key, and status (Allocated/Picked/Staged).

9. Roll Media: Thickness-Based Remaining Length Calculator (Required)

9.1 Roll Item Master Fields

- Material thickness (caliper) stored in mils (preferred) with conversion to inches.
- Core diameter (inches) with default (e.g., 3.0 in) and per-item override.
- Winding/pack factor (default 0.95, configurable).
- Roll width (inches).

9.2 Roll Unit Fields (Per Physical Roll)

- Unique Roll Unit ID (QR).
- Current remaining length (stored truth).

- Optional: starting length, last measured build/OD value + timestamp.

9.3 Update Methods (UI)

- Manual entry: set remaining length directly.
- Outer Diameter (OD): user enters OD; system calculates remaining length.
- Build thickness: user enters material build from core outward (e.g., 2.03 in); system calculates remaining length.

9.4 Audit Requirements

- All roll updates must be logged: old value, new value, method, and measurement entered.

10. Core Workflows (Functional Requirements)

10.1 Receiving / Check-In

- Receive into a specific location/bin.
- Capture vendor, quantity, unit cost (recommended), received date, optional lot/expiry, and attachments (packing slip photo/PDF).
- Support quick receive (no PO) in Phase 1; receiving against PO in Phase 2.
- Generate internal unit IDs and labels where required (roll units).

10.2 Allocations (Required)

- Inventory can be allocated (reserved) to a Jira job and/or client, reducing available stock immediately.
- Pick lists can pull from available stock or already allocated stock for that job/client.
- Support partial allocations with shortage tracking and purchasing notifications.

10.3 Pick Lists (Material Pull)

- Generate pick list per Jira job with required quantities and locations/bins.
- Scan-to-pick workflow (preferred) with manual override.
- Support staging locations (Picked/Staged) separate from storage bins.

10.4 Consumption / Production Usage

- Consume materials against a job (planned vs actual) and record waste separately (reason-coded).
- Rolls: consumption must reference a specific roll unit; update remaining length.
- Paper: consume by sleeves and/or sheets; support partial sleeve usage.
- Rigid sheets: consume by sheet count in Phase 1 (remnant tracking optional Phase 2).

10.5 Adjustments, Transfers, and Ledger

- Adjustments: increase/decrease/correction with required reason code and audit log (user, timestamp, before/after).
- Transfers: move stock between bins/locations with audit log.

- All inventory state changes must be represented as ledger transactions (append-only).

10.6 Cycle Counts

- Cycle count by location/bin, category, or high-value items.
- Variance calculation and posting with audit history.
- Optional supervisor approval step before posting.

11. Jira Integration (Phase 1)

11.1 Job Data

- Lookup Jira job by issue key.
- Store/preview client name, due date, and job type/category when available.

11.2 Status Updates Back to Jira

- Pick list created.
- Picked/Staged.
- Shortage flagged.
- Materials consumed.

12. BOM / Templates (Recommended)

BOM/Templates standardize common material requirements for repeat job types and reduce missed materials.

12.1 Simple BOM Templates (Phase 1)

- Fixed quantities (e.g., 1 pair gloves, 1 sleeve paper, 1 sheet ACM).
- Apply to a Jira job to generate an editable required materials list prior to allocation.

12.2 Formula BOM Templates (Phase 2 Optional)

- Quantities scale based on dimensions/qty and waste factors.
- Supports roll-media area-based planning and standardized waste assumptions.

13. Reporting (Phase 1)

- On-hand and available (on-hand minus allocated) by item and by location/bin.
- Low stock report based on available quantity (not just on-hand), filterable by category.
- Inventory movement ledger (receipts, transfers, allocations, consumption, adjustments).
- Material usage by Jira job (planned vs actual) and by client.
- Waste/scrap reporting by reason code and category.
- Vendor receipts/spend summary and last-cost by vendor.

14. Non-Functional Requirements

- Mobile-friendly UI for receiving, picking, consuming, and counting (phone/tablet).
- Fast search and scan response (target: under 2 seconds for common queries).
- Daily backups and documented restore procedure.
- RBAC permissions and immutable audit log entries.
- Transaction-ledger architecture (do not rely on overwriting quantities without recording a transaction).

15. Future-Proof Features (Designed Now, Build Later)

Design these into the schema and workflows now, even if built later:

- Hold/Quarantine status.
- Substitutions/equivalencies.
- Staging locations and job totes.
- Reorder logic based on available (on-hand minus allocated).
- Lightweight Purchase Orders (POs) and receiving against PO.
- Lot/expiry tracking per item.
- Client-owned/client-specific inventory.
- Remnant tracking (rigid and roll drops).
- BOM templates with formulas and standardized waste factors.
- Job cost rollup (planned vs actual) including waste costs.
- Offline scan queue for shop-floor tablets.
- QuickBooks export/sync options.
- Category-based permission scoping (optional).

16. Delivery Phases (Recommended)

16.1 Phase 1 (Railway Cloud MVP)

- Custom DB schema + migrations (with org_id/tenant_id columns for future).
- Item master + categories + UOM conversions (paper sleeves/sheets; roll remaining length).
- Vendors + item-vendor pricing.
- Locations/bins + transfers + staging locations.
- Barcode strategy: vendor barcode mapping + internal SKU labels + unit labels for rolls; label printing.
- Receiving + allocation required + pick lists (scan-to-pick) + consumption + waste reason codes.
- Cycle counts + audit trail + core reporting (category filters included).
- Jira job lookup + status updates back to Jira.
- RBAC integration via separate Auth/RBAC module; inventory checks capabilities only.

16.2 Phase 2 (Enhanced Ops)

- Purchase Orders (POs) + receiving against PO + ETAs/backorders.
- Substitutions/equivalencies and client-owned inventory.
- Lot/expiry tracking for select items.
- Remnant tracking (rigid and roll drops).
- Approval workflows for high-value adjustments (optional).

16.3 Phase 3 (AWS/Azure Migration or Gnosis Platform Integration)

- Keep UI in Vue as part of Gnosis/iDelta or a standalone SPA while preserving API compatibility.
- Move the same containerized services and PostgreSQL schema to Azure or AWS; keep endpoint contracts stable.
- Advanced job costing and dashboards; offline scanning queue; optional QuickBooks integration.

Additions in v1.4

This version adds: (1) guidance on producing a developer build spec, and (2) bin codes/bins labels plus a label printing feature set.

Developer Build Spec

Yes, a developer build spec will be very helpful. The requirements document defines what the system must do; the build spec removes ambiguity about how to implement it, which reduces rework and keeps Phase 1 on track—especially with custom tables, a transaction ledger, scanning workflows, and Jira integration.

Build Spec Deliverables

- Database schema: tables, columns, keys, indexes, and tenant/org_id strategy.
- Ledger transaction types and invariants (receive, allocate, pick, consume, transfer, adjust, cycle count, roll measure).
- REST API endpoints with request/response payloads and error codes.
- Capability matrix: which actions require which capabilities (RBAC handled by separate Auth/RBAC module).
- UI screen list with field-level requirements and scan flows (mobile first).
- Jira integration map: fields used, write-back method (comment/custom field/transition), retry and failure handling.
- Label templates: bin labels, unit labels (rolls), and optional staging/job labels; supported sizes and output formats.
- Test plan: acceptance tests for the critical shop-floor workflows (receive -> allocate -> pick -> consume -> report).

Recommendation: treat the Railway MVP as a cloud-agnostic product from day one: containerized API, Vue frontend, PostgreSQL, documented REST/OpenAPI contracts, migrations, and provider-neutral adapters for storage, jobs, labels, and integrations.

Bin Codes, Bin Labels, and Label Printing

Yes—this system should generate bin codes/labels and include a print-label feature. Bin labels make receiving, picking, and cycle counting dramatically faster and reduce picking errors.

Bin Codes

- Each bin must have a unique Bin Code per org/tenant.
- Bin Code can be auto-generated (recommended) with optional manual override.
- Suggested format: <LOC>-<AISLE>-<RACK>-<SHELF>-<BIN> (human readable) plus a short unique ID for scanning, e.g., NP-01-A2-R3-S1-B04 / BIN-8H3KQ2.
- System must prevent duplicates and maintain a lookup table for scanned codes.

Bin Labels (Content)

- QR code (preferred) and/or Code128 barcode encoding the Bin Code or Bin ID.
- Location name and bin path (Aisle/Rack/Shelf/Bin).
- Optional: zone (e.g., 'Flatbed Room', 'Shipping'), and notes (e.g., 'Paper Only').
- Optional: a small 'Do not store' / 'Quarantine' marker if bin is restricted.

Label Printing Feature

- Print bin labels from the Location/Bin management screen (single print and batch print).
- Print unit labels during receiving (required for rolls and any unit-tracked items).
- Optional: print staging/job tote labels when a pick list is created (Job Key + Client + date + QR).
- Batch print from a receiving session (e.g., print labels for 20 received rolls at once).

Label Output Formats (Implementation Options)

- PDF label sheets rendered in browser (works with most printers).
- Optional: ZPL output for Zebra label printers (faster and more reliable for shop-floor labels).
- Support configurable label sizes (common: 2x1, 2x2, 3x1).

Acceptance Criteria

1. Admin/Inventory Manager can create locations and bins; the system generates unique Bin Codes.
2. User can print a bin label that scans back into the correct bin record.
3. Receiving flow can print unit labels for each received roll; scanning the label opens the roll unit record.
4. Pick flow supports scanning bin label then scanning unit label to confirm correct pull.
5. All print actions are logged (who printed, what template, timestamp) for troubleshooting.

Additions in v1.5

This version revises the deployment approach from a WordPress plugin MVP to a Railway-hosted, cloud-agnostic Gnosis product that can later move to AWS, Azure, or the broader Gnosis/iDelta platform without a rewrite.

17. Cloud-Agnostic Product Architecture (Railway MVP)

The system should be built as a standalone Gnosis product hosted initially on Railway, but structured so hosting can later move to AWS or Azure. Railway should be treated as the first deployment environment, not the permanent application architecture.

17.1 Recommended Application Stack

- Frontend: Vue 3 or Nuxt-based web application for desktop and mobile shop-floor screens.
- Backend API: Node.js service using a structured framework such as NestJS, Fastify, or Express with TypeScript.
- Database: PostgreSQL as the primary system of record.
- Background Worker: separate worker service for Jira sync, label generation batches, reports, notifications, and future QuickBooks/export jobs.
- Cache/Queue: Redis or a Postgres-backed queue in Phase 1; Redis can be added when job volume increases.
- Storage: S3-compatible object storage adapter for packing slips, attachments, generated labels, exports, and future PDFs.
- Auth/RBAC: separate Auth/RBAC module or service; inventory reads permissions/scopes and does not own identity management.
- API Contract: REST/OpenAPI first, with stable request/response schemas so future Vue/Gnosis clients can reuse the same endpoints.

17.2 Railway Deployment Topology

- Service 1: Web frontend (Vue/Nuxt build or static frontend service).
- Service 2: API backend.
- Service 3: Background worker.
- Service 4: PostgreSQL database.
- Optional Service 5: Redis queue/cache.
- Use environment variables for all configuration and secrets.
- Use private/internal networking for service-to-service communication wherever possible.
- Use Dockerfiles or portable build configuration so the same services can be moved later to AWS or Azure.
- Run database migrations as a controlled pre-deploy/release step, never manually in production.

17.3 Cloud-Agnostic Rules

- No Railway-specific URLs, file paths, or IDs should be stored as business data.
- All secrets and service URLs must come from environment variables or a secrets manager.

- All deployable services must be containerized or otherwise buildable from the same repository in CI/CD.
- Use PostgreSQL-standard schema and migrations; avoid provider-specific database features unless isolated behind migrations/adapters.
- Use an object storage adapter interface rather than hard-coding one cloud provider.
- Use a notification adapter for email/SMS/Slack so providers can change later.
- Use a Jira integration adapter so Jira-specific API behavior stays isolated from core inventory logic.
- Use an explicit org_id/tenant_id on all core tables even if NuePrint is the only org at launch.
- Maintain OpenAPI documentation and version endpoints under /api/v1 to preserve compatibility over time.

17.4 Migration Path: Railway to AWS or Azure

6. Export or replicate PostgreSQL database and object storage data.
7. Deploy the same frontend, API, and worker containers to AWS or Azure.
8. Move environment variables/secrets to the target cloud secrets system.
9. Point DNS to the new frontend/API.
10. Run smoke tests: login/RBAC, receive material, allocate, print label, pick, consume, Jira update, and report generation.
11. Keep Railway live temporarily as a rollback environment until production validation is complete.

18. Updated Developer Build Spec Direction

The developer build spec should now be written for a cloud-hosted Gnosis product instead of a WordPress plugin. The MVP can run on Railway, but the spec must assume future movement to AWS, Azure, or Gnosis/iDelta infrastructure.

18.1 Build Spec Additions

- Repository structure: frontend, api, worker, shared types/schemas, infrastructure scripts, and database migrations.
- Deployment files: Dockerfiles or build configuration for frontend/API/worker services.
- Environment variable matrix: local, staging, production.
- API schema: OpenAPI document for inventory, labels, Jira, reports, and RBAC checks.
- Database migrations and seed data: categories, reason codes, default locations, default label templates.
- Provider adapters: storage, email/notifications, queue, label rendering, Jira integration.
- CI/CD plan: test, build, migrate, deploy, smoke test.
- Backup and restore plan: PostgreSQL backups, object storage retention, restore drills, and export plan for cloud migration.

19. Reference Notes

Railway-specific deployment notes should be verified against Railway documentation during implementation. Current planning assumes Railway supports deployable services, environment variables, private networking between services, custom Dockerfile/build configuration, PostgreSQL services, and volume/database backup features.

Additions in v1.6

This version adds the missing print-shop operating model for NuePrint as the first use case, plus a scalable repository structure and Codex engineering instructions for production-ready implementation.

20. NuePrint Print-Shop Operating Model Requirements

The inventory system must support print-production realities rather than treating every SKU as a generic quantity. Each item category has its own fields, tracking behavior, allocation rules, and reporting needs.

20.1 Category-Specific Inventory Rules

Each top-level inventory category must support required fields, optional metadata, tracking behavior, and category-specific UX. Categories must remain configurable so NuePrint can add new product lines without schema rewrites.

Category	Required Inventory Behavior
Roll Media / Roll Materials	Unit-tracked by physical roll. Must support width, starting length, remaining length, material thickness/caliper, core diameter, finish, adhesive type, printer compatibility, laminate compatibility, waste factor, vendor, cost, and preferred use rule. Roll labels must identify the exact roll unit.
Rigid / Hard Substrates	Tracked by sheets, partial sheets, and future remnants. Must support substrate type, thickness, sheet size, color/face, indoor/outdoor suitability, printer compatibility, cut method, and full-sheet/partial-sheet policy.
Paper Products	Tracked by sleeves and sheets. Must support sheet size, paper type, weight in lb/gsm, caliper/thickness, coating/finish, sheets per sleeve, sleeves per carton, printer compatibility, and optional grain direction.
Apparel / Garments	Tracked by variant. Brand, style, garment type, color, size, decoration compatibility, vendor SKU, client-owned status, and substitution rules are required. Size/color variants must not be collapsed into one quantity.
Inks	Tracked by machine compatibility, ink type, color channel, container size, lot, expiration, opened date, installed vs spare status, cost per liter/ml, and optional estimated cost per ft2/in2.
Maintenance Items	Tracked by machine compatibility, kit/item type, recommended replacement interval, criticality, min/max stock, last replaced date where applicable, and production-stopping flag.
Consumables / Packaging / Accessories	Tracked by each, box, case, pack, or roll. Must support consumable type, normal usage per job type, criticality, reorder point, par level, and optional client/job allocation.

20.2 Machine / Work Center Model

Machines must be modeled as work centers, not just inventory assets. Materials, inks, maintenance kits, and consumables should be linkable to compatible machines/work centers.

- Machine/work center fields: name, type, model, serial number if applicable, location, active/inactive status, supported material categories, max width/max sheet size, compatible ink sets, compatible maintenance kits, and default waste factors.
- Example work center types: roll printer, flatbed printer, plotter, laminator, cutter/router, laser, DTG/DTF printer, heat press, embroidery, finishing table, shipping/packaging station.
- Job material selection must be able to filter by machine compatibility so users do not allocate incompatible stock.

20.3 Product and Jira Job Specification Fields

The Jira connection must include enough production detail to calculate and allocate material. Jira may remain the job source of truth, but inventory must store a normalized job snapshot for calculations, pick lists, and audit history.

- Required job fields: Jira issue key, client, product type, quantity, finished size, flat size where applicable, due date, rush flag, assigned work center/machine, material requested, finish options, sides, cut method, bleed/waste allowance, and file/attachment link reference.
- Optional job fields: install required, shipping method, packaging requirements, client-owned material flag, proof approval status, and production notes.
- If a required field is missing, the system must flag the job as incomplete for material calculation rather than guessing silently.

20.4 Formula BOM / Material Calculation Requirements

The system must support both fixed BOM templates and formula-based BOM templates. Formula BOMs are required for realistic print production because material usage depends on dimensions, quantity, waste, finishing, and machine rules.

- Banner formula example: width x height + waste factor -> banner roll length, ink estimate, grommets, hem tape, weld/stitch labor reference, and packaging.
- Sticker formula example: sticker size x quantity + spacing/waste -> printable vinyl, laminate, cut path estimate, transfer tape if applicable, and sheet/roll yield.
- Rigid sign formula example: finished size x quantity -> sheet count, sheet utilization, full/partial sheet rule, cut method, and remnant creation if remaining area is usable.
- Apparel formula example: garment variant x quantity -> blank allocation plus DTF/DTG/embroidery consumables as configured.
- Paper formula example: finished size, parent sheet size, imposition/yield, sides, and spoilage factor -> sheets and sleeves consumed.

20.5 Allocation Rules by Material Family

Material Family	Allocation Rule
Rolls	Allocate a specific roll unit and reserved length. Available length must decrease immediately. Picking and consumption must reference the roll unit.
Rigid sheets	Allocate full sheets, partial sheets, or remnants. System must warn when the job forces full-sheet billing/usage due to inefficient layout.
Paper	Allocate by sheet count or sleeve count. Partial sleeve usage must be supported.
Apparel	Allocate exact brand/style/color/size variants. Shortages must be reported by variant.
Inks	Normally not job-allocated unless estimating job cost. Actual consumption may be posted by machine, job, or batch depending on workflow.
Maintenance items	Normally consumed by machine/service event, not by client job, unless a specific job requires unusual maintenance.
Consumables	Can be general-use or job-allocated depending on item setup. Job-critical consumables should appear on pick lists.

20.6 Remnants and Partial Inventory

The data model must support remnants/partials even if the complete workflow is phased in later. Remnants prevent waste and make rigid/roll inventory more accurate.

- Remnant fields: parent unit/item, dimensions (width/height or width/length), usable area, material family, location/bin, status, created from job/cut reference, and minimum keep threshold.
- System must allow auto-scrap below a configured size threshold and create a ledger transaction for the scrap event.
- Rigid remnants must be searchable during allocation and pick list generation.
- Roll offcuts may be tracked as remnant units if the shop chooses to keep them.

20.7 Purchasing and Reorder Logic

Purchasing logic must use available stock rather than raw on-hand stock.

- Available = on-hand - allocated - quarantine - expired/scrap + incoming PO quantity where applicable.
- Item/vendor fields: preferred vendor, alternate vendors, vendor SKU, vendor URL, pack size, MOQ, lead time, last cost, average cost, effective dates, and freight notes.
- Reorder fields: min stock, max stock, reorder point, reorder quantity, par level, and criticality.
- Reports must show low stock, suggested purchase quantity, vendor, estimated cost, and open/backordered PO quantities.

20.8 Receiving Quality Control

Receiving must support inspection and discrepancy tracking, not only quantity entry.

- Receiving statuses: received, pending inspection, damaged, wrong item, short shipment, quarantine, returned to vendor, and available.
- Receiving fields: vendor, packing slip, received by, received date, item condition, expected quantity, received quantity, discrepancy notes, photo uploads, lot/expiration where applicable, and vendor claim/reference number.
- Quarantined material must not be available for allocation or picking until released by an authorized user.

20.9 Stock Status Model

All inventory units/quantities must use a formal status model so reports and availability do not become ambiguous.

- Core statuses: available, allocated, picked, staged, consumed, quarantine, damaged, scrap, expired, returned, client-owned, on order, and backordered.
- Statuses must affect availability rules consistently across reports, allocation screens, pick lists, and reorder logic.
- Status changes must be ledgered and audited.

20.10 Count Controls

- Support blind count mode, count freeze by bin/category, recount thresholds, variance approval thresholds, count sessions, and scanner-based counting.
- Counts may be performed by location, category, machine compatibility, high-value items, low-stock items, or random cycle-count schedule.
- Variance posting requires reason code and audit trail. High-value variances can require approval.

20.11 Print-Shop Reporting Expansion

- Low stock by category and by machine compatibility.
- Allocated inventory by job/client and aging allocations.
- Materials consumed by product line and Jira job.
- Planned vs actual material cost by job.
- Waste by material, machine, operator, and reason.
- Roll remaining length report and rigid remnant report.
- Ink usage by machine and maintenance item usage by machine.
- Apparel stock by brand/style/color/size.
- Vendor price changes, stockout history, inventory valuation, slow-moving/dead stock, and client-owned inventory.

20.12 Import / Seed Data Requirements

The system must include clean import paths because NuePrint already has material, pricing, and product references across multiple categories.

- CSV import for items, categories, vendor pricing, starting inventory, barcode mappings, locations/bins, and machine/material compatibility.

- Bulk validation with an error report before importing.
- Dry-run mode to preview changes before writing data.
- Seed data scripts for local development and demo environments, separated from production data.

20.13 Integration Contract Details

- Define whether Jira is the source of truth for job creation. Default assumption: Jira creates/owns jobs; inventory stores a synced job snapshot.
- Define which Jira issue types count as production jobs and which fields map to client, product type, quantity, dimensions, due date, material, finish, and status.
- Inventory must support webhook-triggered sync, manual re-sync, failed sync queue, retry logic, and clear user-facing sync errors.
- Inventory write-back to Jira may use comments, custom fields, status transitions, or a combination. The method must be configurable.

20.14 Acceptance Criteria / User Stories

12. Given a new roll is received, when the user checks it in, then the system creates a roll unit, stores vendor/cost/location, prints a QR label, and records a receive ledger transaction.
13. Given a Jira job needs 20 ft of 54-inch vinyl, when the user allocates material, then available length decreases and the selected roll is reserved to that job.
14. Given a production user scans a picked roll, when they enter measured build thickness or OD, then the system recalculates remaining length and writes an audited roll measurement transaction.
15. Given an apparel job needs 12 black XL shirts and only 10 are available, when the user allocates blanks, then the system allocates 10 and flags a shortage of 2 for that exact variant.
16. Given a rigid sign is cut from a 4x8 sheet, when the remaining piece is above the keep threshold, then the system creates a remnant record with dimensions, location, and parent sheet reference.
17. Given receiving finds damaged or incorrect material, when the user marks the receipt as quarantine or returned, then that stock is not available for allocation or picking.
18. Given a bin count variance exceeds the approval threshold, when the count is submitted, then the system requires approval before posting the adjustment.
19. Given Jira is temporarily unavailable, when inventory attempts a write-back, then the system queues the sync, shows a clear failed-sync state, and allows manual retry.

21. Scalable Repository and Folder Structure

The application must be structured as a cloud-agnostic, modular product that can start on Railway and later move to AWS, Azure, or the broader Gnosis/iDelta platform. The folder structure must make it easy to add product lines and features without creating giant files or tightly coupled modules.

21.1 Repository Strategy

- Use a monorepo unless there is a strong reason not to. This keeps frontend, API, workers, shared contracts, and domain logic versioned together.
- Separate applications from reusable packages. Apps can be replaced later; shared contracts/domain logic should survive hosting and frontend changes.
- Use feature modules for inventory workflows (items, receiving, allocation, picking, consumption, counts, reports, labels, work centers, purchasing).
- Keep integrations isolated behind adapters (Jira, label printer, storage, email/notifications) so cloud/vendor changes do not affect core inventory logic.

21.2 Recommended Folder Structure

```
gnosis-inventory/
  apps/
    web/
      src/
        app/
        modules/
          inventory/
            items/
            receiving/
            allocations/
            picking/
            consumption/
            counts/
            reports/
            labels/
            work-centers/
            purchasing/
          jobs/
          auth/
          settings/
        shared/
          api/
          auth/
          ui/
          validation/
    api/
      src/
        main.ts
        app.ts
        config/
        modules/
          inventory/
            items/
            categories/
            units/
            receiving/
            allocations/
            picklists/
            consumption/
            counts/
            reports/
            labels/
            work-centers/
```

```
          purchasing/
        jobs/
        auth/
        audit/
        integrations/
          jira/
          label-printers/
          object-storage/
      middleware/
      lib/
    db/
      migrations/
      seeds/
  worker/
    src/
      jobs/
      queues/
      processors/
      integrations/
packages/
  domain/
    src/
      inventory/
      jobs/
      pricing/
      shared/
  contracts/
    openapi/
    schemas/
  database/
  config/
  logger/
  testing/
  ui/
infra/
  railway/
  docker/
  terraform/
  aws/
  azure/
docs/
  requirements/
  architecture/
  decisions/
  runbooks/
  api/
scripts/
tests/
  e2e/
  integration/
```

21.3 Feature Module File Pattern

Each module should use small, purpose-specific files. Naming can adjust by framework, but the separation of responsibilities must remain.

```
apps/api/src/modules/inventory/items/
  items.routes.ts          # HTTP route registration only
```

```
items.controller.ts      # request/response orchestration only
items.service.ts         # business logic
items.repository.ts      # database access only
items.schemas.ts         # request/response validation schemas
items.types.ts           # local TypeScript types if needed
items.permissions.ts     # capability constants/check helpers
items.events.ts          # domain event names/payloads if needed
items.test.ts            # unit tests for service-level behavior
```

- Controllers must stay thin and never contain core inventory math or ledger rules.
- Services contain business rules and call repositories, validators, ledger services, and integration adapters.
- Repositories contain database queries and no business decisions.
- Schemas/contracts define what the API accepts and returns. API contracts should be kept versioned.

21.4 File Length and Modularity Policy

- No hand-authored source file may exceed 300 lines. Target 150-220 lines where practical; split proactively once a file approaches 250 lines.
- Do not solve the 300-line limit by hiding unrelated behavior in helper files. Split by responsibility: routes, controller, service, repository, schemas, types, permissions, tests, and utilities.
- Generated files should be avoided when possible. If a required generated artifact exceeds 300 lines, it must be clearly marked as generated and not hand-edited. Hand-authored production code remains subject to the 300-line limit.
- Large config, seed, schema, and migration files should be split by domain/module where tooling allows.

21.5 Architectural Boundaries

- Inventory domain logic must not depend on Railway, AWS, Azure, or a specific frontend framework.
- Business rules must not live only in the frontend. Server-side API must enforce RBAC, org/tenant boundaries, validation, and inventory invariants.
- All inventory quantity changes must go through the ledger service; direct quantity mutation is prohibited except through controlled projections/cached views.
- All integration-specific behavior must go through adapters. Example: JiraAdapter, LabelPrinterAdapter, StorageAdapter, NotificationAdapter.
- Cross-module imports should flow inward toward domain packages and shared contracts, not randomly between feature modules.

22. Codex Engineering Instructions

These instructions are intended to be given to Codex or any code-generation/development agent working on the project. They are mandatory unless explicitly overridden by a project maintainer.

22.1 Production-Ready Requirement

- All generated or edited code must be production-ready for the requested scope. No stubs, no fake implementations, no TODO placeholders, no empty handlers, and no code paths that pretend to work but are incomplete.
- If a feature is out of scope, do not add a broken placeholder. Document it as out of scope and leave the application working.
- Every new API endpoint must include validation, RBAC/capability checks, tenant/org boundary checks, error handling, logging/audit behavior where applicable, and tests.
- Every inventory-changing action must create a ledger/audit record and use database transactions where consistency matters.

22.2 300-Line File Limit

- Do not create or leave hand-authored source files longer than 300 lines.
- Before adding code to a file near the limit, refactor into smaller files by responsibility.
- When modifying an existing file over 300 lines, first split it into smaller cohesive files, then make the requested change.
- Do not bypass the limit by minifying code, compressing formatting, or combining unrelated logic into generic utility files.

22.3 Required Completion Standard

- A task is not complete until the feature works end-to-end for its defined scope.
- Add or update tests for the behavior changed. At minimum: unit tests for domain logic, integration tests for API/DB behavior, and E2E tests for critical scanner workflows when UI is involved.
- Run lint, typecheck, tests, and database migration checks before handing off. If a command cannot be run, state why and what remains unverified.
- Do not hard-code secrets, org IDs, vendor IDs, printer names, or environment-specific URLs. Use config and environment variables.

22.4 Inventory-Specific Guardrails

- Never adjust stock by overwriting quantity without a ledger transaction.
- Never allow allocated, quarantined, expired, damaged, or scrapped stock to appear as available.
- Never trust frontend-calculated inventory values for final writes. Server-side services must recalculate and validate.
- Use row-level locking or equivalent transaction safeguards where concurrent allocation/consumption can race.
- Roll remaining length, paper sleeve/sheet conversion, remnant creation, and allocation logic must be tested with edge cases.

22.5 Documentation and Handoff

- Update docs when adding or changing architecture, environment variables, API contracts, migration behavior, or operational workflows.

- Each meaningful architecture choice should be captured in docs/decisions as an ADR when it affects future migration or module boundaries.
- Each module should include concise README notes only when needed; do not create bloated documentation files that repeat code.
- When handing off work, summarize changed files, tests run, migrations added, and any remaining risks. There should be no hidden gaps.

23. Developer Build Spec Additions for v1.6

Before implementation begins, the developer build spec should be expanded beyond the requirements document with the following concrete artifacts.

- Database schema by module: items, categories, vendors, item-vendor pricing, units, bins, ledger, allocations, picklists, remnants, work centers, receiving QC, purchasing, labels, jobs, audit, and integrations.
- OpenAPI contract for all API endpoints, with versioned request/response payloads and error codes.
- Ledger transaction type list with required fields and invariants.
- Screen-by-screen UX requirements for receiving, labels, allocation, pick list, consumption, roll measurement, counts, reports, and admin settings.
- Formula BOM examples and test fixtures for banners, stickers, rigid signs, paper products, apparel, inks, and consumables.
- Migration/runbook: Railway deployment, local dev setup, backups/restore, environment variables, and AWS/Azure portability notes.
- Codex prompt/instruction file stored in repo, preferably docs/architecture/codex-engineering-instructions.md or .codex/instructions.md.